

TECNOLOGÍAS DE LA INFORMACIÓN EN LINEA

SISTEMAS EMBEBIDOS

3 créditos



Profesor:

Ing. Freddy Carrera Sánchez, Msc.

Titulaciones	Semestre
Ingeniero en Tecnologías de la Información	Sexto

Tutorías: El profesor asignado se publicará en el entorno virtual de aprendizaje (online.utm.edu.ec), y sus horarios de conferencias se indicarán en la sección **CAFETERÍA VIRTUAL**.

PERÍODO ABRIL – AGOSTO 2026

Índice

Tabla de contenido

Unidad 2. EL HARDWARE DE UN SISTEMA EMBEBIDO.....	4
Tema 2.1: Hardware de Sistemas Embebidos.....	5
2.1.1 Organización de un microcomputador	5
2.1.2 Desarrollo de hardware embebido.....	8
Tema 2.2: Arquitectura de hardware embebido.....	12
2.2.1 Arquitectura de Microcontroladores	12
Según la conexión de componentes.....	12
Según complejidad del conjunto de instrucciones.	13
Según el tamaño de palabra (ancho de bus).	14
Según especificaciones de diseño o familia.....	14
2.2.2 Ejemplos de Hardware: Interfaces GPIO y seriales.....	16
Ejemplos de microcontroladores.....	16
Entornos de programación de hardware embebido.	18
Comunicación en paralelo y serie: GPIO.....	20
Interfaces seriales de comunicación.....	21
Tema 2.3: Diseño lógico de hardware embebido	26
2.3.1 Diseño con lenguajes de descripción de hardware	26
2.3.2 Descripción estructural, de flujo de datos y de comportamiento.	27
Microcontrolador a diseñar.....	27
Implementación de Memoria de Programa	30
Implementación de Memoria de Datos.....	32
Implementación de Puertos de E/S.	33
2.3.3 Modelo, entorno y mecanismo de simulación	34
Herramientas & Software de la asignatura	37
Bibliografía.....	38

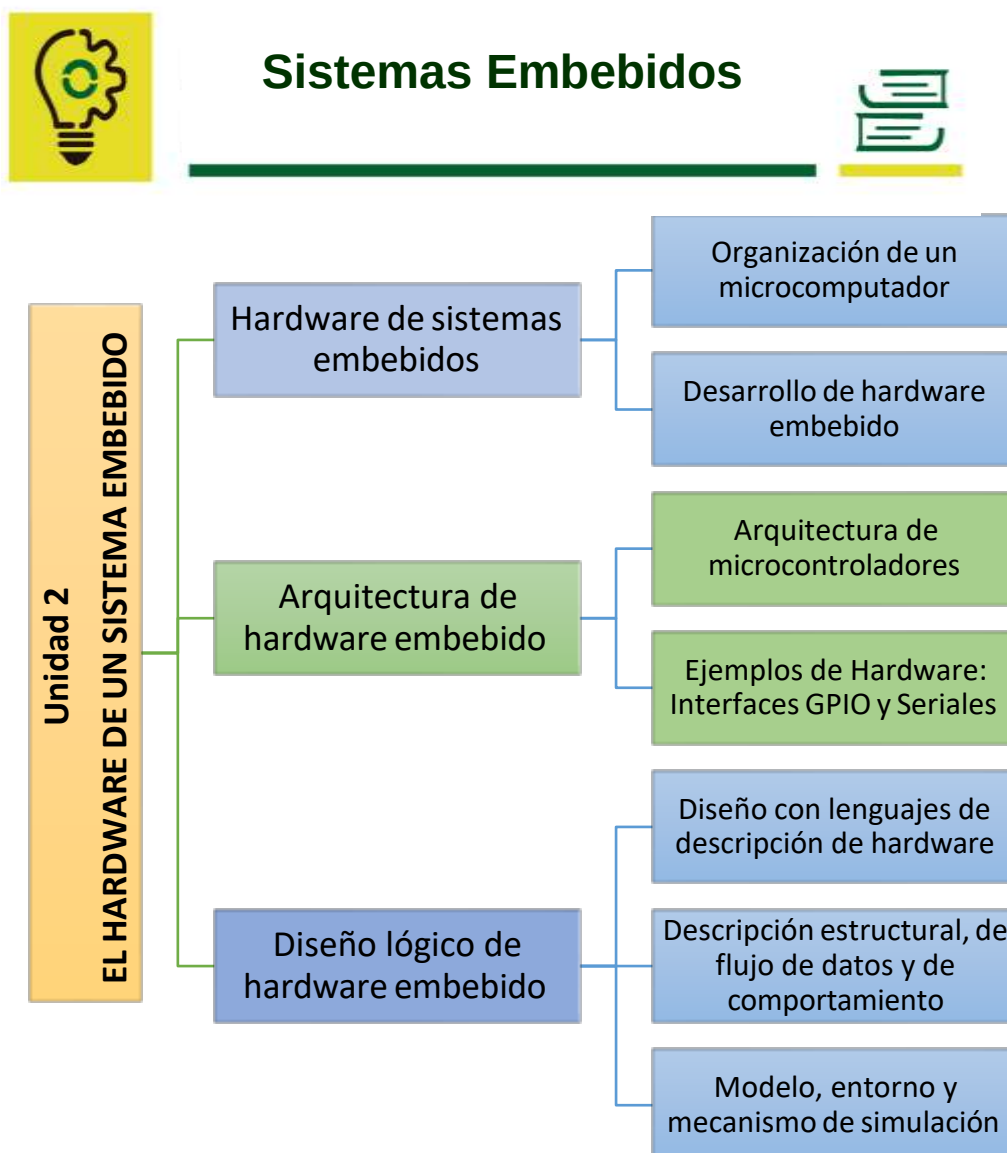


Organización de la lectura para el estudiante por semana del compendio

Semanas	Paginas
Semana 4	Páginas 4 –11
Semana 5	Páginas 12 – 16
Semana 6	Páginas 16 –25
Semana 7	Páginas 26 –36
Semana 8	Repaso

Resultado de aprendizaje de la asignatura

Implementar un sistema embebido básico mediante el uso de herramientas de prototipado de hardware para el control de dispositivos que soporten soluciones en diferentes ámbitos.



Unidad 2. EL HARDWARE DE UN SISTEMA EMBEBIDO

Resultado de aprendizaje de la unidad: Integrar sistemas de hardware usando componentes eléctricos y electrónicos básicos para la implementación de manera segura y óptima de un sistema embebido.

Tema 2.1: Hardware de Sistemas Embebidos



En este tema veremos cómo se estructura y cómo puede diseñarse y desarrollarse el hardware para un sistema embebido.

2.1.1 Organización de un microcomputador

Para entender la estructura y diseño del hardware de un Sistema Embebido, hay que revisar la estructura de un microcomputador. La configuración de hardware mínima de un sistema de microcomputadora tiene tres componentes fundamentales:

- Unidad Central de Procesamiento (CPU)
- La memoria del sistema
- Alguna forma de Interfaz de Entrada / Salida (E/S).

Estos componentes están interconectados por múltiples conjuntos de líneas agrupadas según sus funciones, y denominadas globalmente buses del sistema. Un conjunto adicional de componentes proporciona la energía necesaria y la sincronización de tiempo para el funcionamiento del sistema (Figura 1).

Los componentes de una microcomputadora se pueden implementar de diversas formas: Pueden implementarse con múltiples chips a nivel de placa; o integrarse en un solo chip, en una estructura llamada microcomputadora en un chip, o simplemente un microcontrolador.

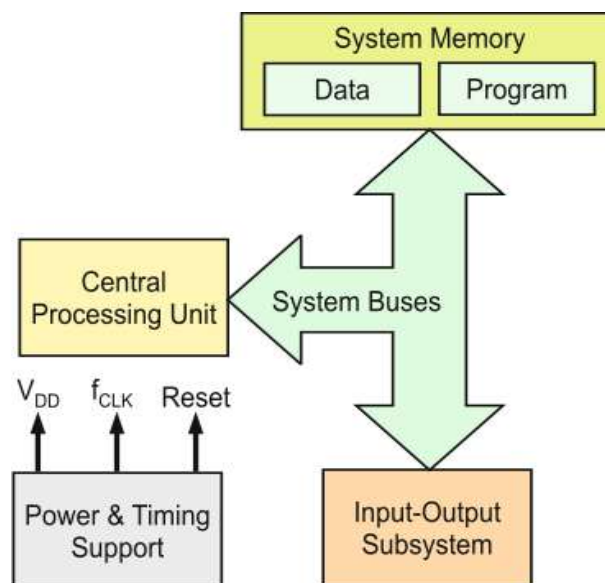


Figura 1. Arquitectura general de un microcomputador (Jiménez, Palomera, & Couvertier, 2014).

Hoy en día, la mayoría de los sistemas embebidos se desarrollan alrededor de microcontroladores. Comparando la estructura de hardware de un sistema embebido de la unidad anterior con la Figura 1, notamos que todos los componentes de hardware (menos CPU y memoria), se condensaron en el bloque del subsistema de entrada/salida.

Esto nos confirma que los sistemas embebidos son de hecho microcomputadoras.

Analicemos brevemente la función de estos componentes:

- **Unidad central de procesamiento (CPU):** Forma el corazón del sistema del microcontrolador. Recupera instrucciones de la memoria del programa, las decodifica, y opera con datos y / o dispositivos periféricos en el subsistema de entrada-salida.
- **Memoria del sistema:** Es el lugar donde se almacenan los programas y los datos para que la CPU pueda acceder a ellos. Dos tipos de elementos de memoria dentro del sistema:
 - Memoria de programas, que almacena programas en forma de secuencia de instrucciones.
 - Memoria de datos, que almacena los datos que se utilizarán con los programas.
- **Subsistema de entrada / salida:** También llamado Subsistema de periféricos, incluye todos los componentes o periféricos que permiten a la CPU intercambiar información con otros dispositivos, sistemas o con el mundo externo.
- **Buses del sistema:** Es el conjunto de líneas que interconectan la CPU, la memoria y el subsistema de E/S. Los grupos de líneas en los buses del sistema realizan diferentes funciones: Bus de direcciones, bus de datos y bus de control.

Microcontroladores vs. Microprocesadores.

A lo largo de esta asignatura se habla mucho de los Microprocesadores y los Microcontroladores. Con el fin de distinguir claramente la diferencia entre ambos, veamos sus características:

- **Unidad Microprocesador (MPU):** Contiene fundamentalmente una CPU de propósito general en su matriz. Para desarrollar un sistema con MPU, el resto de componentes se implementan externamente. Poseen una arquitectura optimizada

para mover código y datos de la memoria externa al chip, y la inclusión de elementos arquitectónicos para acelerar el procesamiento. Representan el tipo más poderoso de componente de procesamiento disponible para un microcomputador. La mayoría de los sistemas embebidos pequeños no necesitan una potencia de este tipo.

- Unidad Microcontrolador (MCU):** Se desarrolla utilizando un núcleo de microprocesador o una CPU, generalmente menos compleja que la de una MPU. Esta CPU básica está rodeada de memoria de ambos tipos (programa y datos) y varios tipos de periféricos, todos ellos integrados en un solo circuito integrado o chip: Esta combinación es lo que llamamos **microcontrolador**. Permite implementar aplicaciones completas que requieren solo una cantidad mínima de componentes externos o, en muchos casos, solo utilizan el chip MCU. Por eso se suelen denominar **computadoras-en-un-chip** (
- Figura 2).

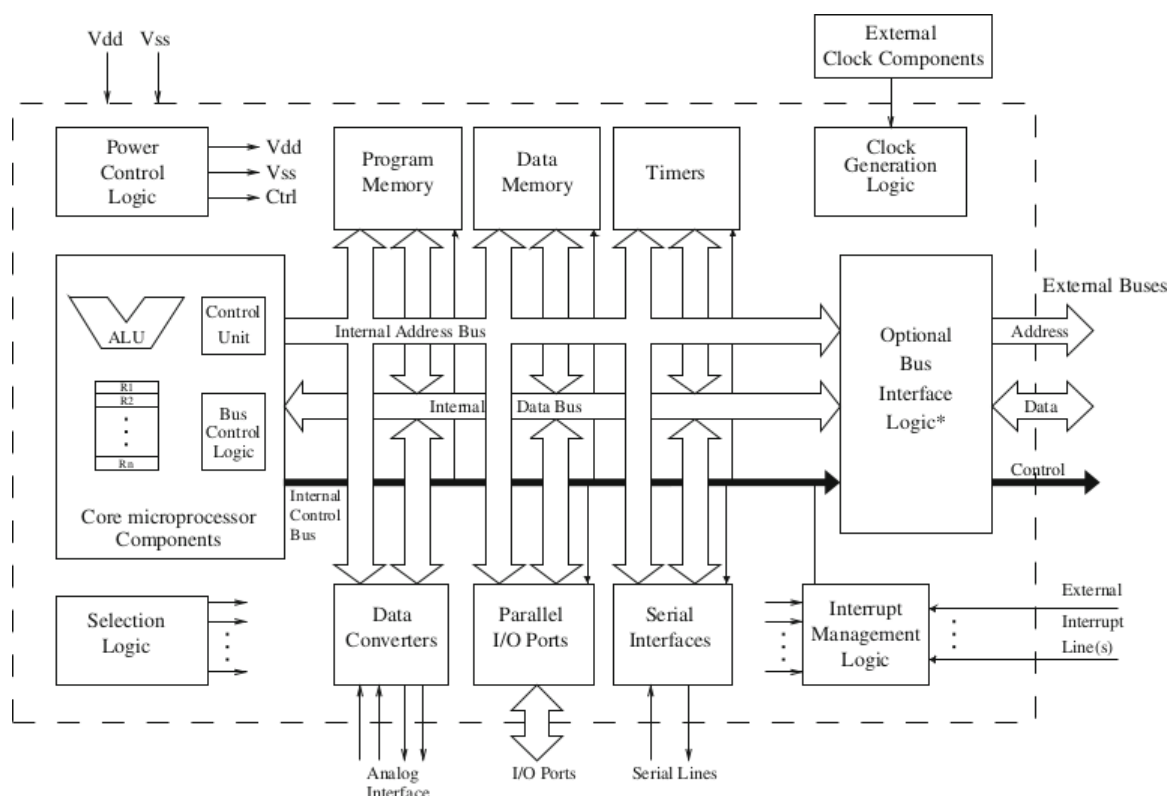


Figura 2. Estructura típica de un microcontrolador (Jiménez, Palomera, & Couvertier, 2014).

Los MCU comparten una serie de características con los microprocesadores de propósito general. Sin embargo, sus componentes arquitectónicos centrales son menos complejos y están más orientados a una aplicación (Jiménez, Palomera, & Couvertier, 2014).



Sabías que: Los microcontroladores se comercializan generalmente como miembros de la familia. Cada familia tiene una arquitectura básica que define las características comunes de todos los miembros (Figura 3).

Company	4-bits	8-bits	16-bits	32-bits
EM Microelectronic	EM6807	EM6819		
Samsung	S3P7xx	S3F9xxx	S3FCxx	S3FN23BXZZ
Freescale semiconductor		68HC11	68HC12	
Toshiba		TLCS-870	TLCS-900/L1	TLCS-900/H1
Texas instruments			MSP430	TMS320C28X
			TMS320C24X	Stellaris line
Microchip		PIC1X	PIC2x	PIC32

Figura 3. Ejemplos de Microcontroladores de distintos fabricantes.

2.1.2 Desarrollo de hardware embebido

El desarrollo completo y confiable de un S.E. requiere que todos los detalles sean evaluados y analizados correctamente. El esfuerzo inicial es significativo, pero es menos costoso y demorado que buscar los errores en producción.

A continuación se detalla una lista de aspectos que sirve como guía para asegurar de que se hayan evaluado todos los aspectos importantes. Esta lista de verificación se puede utilizar como base para una revisión de diseño técnico o para evaluar la exactitud de los diseños de hardware producidos por otros.

1. Definir requerimientos de fuente de alimentación:

- Se deben enumerar todos los voltajes y tolerancias de alimentación, junto con los requisitos de corriente típicos y máximos en el rango de temperatura de cada dispositivo.
- Debe dejar un margen significativo (50 a 100% de exceso de capacidad) entre la carga requerida y la corriente de suministro máxima disponible.
 - Muy importante para aumentar la confiabilidad a largo plazo de la fuente de alimentación.
- Verificar que los voltajes entregados a todos los dispositivos estén dentro de sus especificaciones y que la suma de las corrientes del peor caso pueda ser suministrada con margen

- Mida el consumo de energía real en un circuito prototipo para verificar que esté dentro de los límites esperados.

2. Verificar la compatibilidad del nivel de voltaje:

- Los niveles de voltaje que se producirán en la interfaz de cada tipo de chip en el diseño deben ser compatibles.
- Por dos propósitos: Para que la entrada impulsada interprete el nivel lógico correcto de salida, y para evitar daños potenciales a las entradas del dispositivo.
 - Una puerta de 3 voltios puede tener una especificación de entrada máxima de $V_{cc} + 0,3$ voltios, y activarla con la salida de una puerta lógica de 5 voltios puede dañarla.
- En algunos casos, pueden ser necesarios circuitos de conversión de nivel o de sujeción de voltaje.
 - Algunos dispositivos lógicos de 3 voltios toleran niveles de señal de 5 voltios en algunos de sus pines de entrada, lo que simplifica el diseño.

3. Verificar la corriente de salida vs. carga:

- Se especifican las corrientes de salida máxima I_{OL} e I_{OH} , (OL=salida en bajo, OH=salida en alto), para un voltaje de salida específico V_{OL} y V_{OH} .
- La corriente de carga total que impulsa una salida debe compararse con las entradas y cualquier resistencia que deba impulsar la salida, y debe dejarse un margen suficiente para garantizar un funcionamiento adecuado.
- Algunas salidas lógicas, como las líneas de solicitud IRQ y DMA, utilizan con frecuencia buses de drenaje abierto o colector abierto, que requieren resistencias de tipo pull-up.
- Las entradas no utilizadas deben llevarse a su estado inactivo: ya sea hacia arriba al suministro a través de una resistencia o conectadas a tierra, según corresponda.

4. Unidad de salida de CA (capacitiva) vs. carga capacitiva y reducción de potencia:

- La temporización (pulsos) del dispositivo generalmente se especifica bajo condiciones de carga específicas en las salidas.

- Si la carga capacitiva real en las salidas (entradas lógicas accionadas y la capacitancia del cableado parásito) excede el capacitor de carga especificado en las condiciones de prueba de temporización, las especificaciones de temporización no serán válidas.

5. Verificar el peor caso de condición de temporización:

- Todas las especificaciones de temporización deben evaluarse para detectar posibles violaciones de temporización

6. Determinar si se requiere una terminación de la línea de transmisión:

- El tiempo de subida de la señal y la longitud máxima de la traza (señal) deben evaluarse para determinar si una interconexión de señal debe tratarse como una línea de transmisión, que requiere impedancia constante a lo largo de la traza y terminación para evitar reflejos.
- Para una traza que sea más corta que un sexto de la longitud del borde ascendente o descendente de la señal, el circuito rara vez debe considerarse una línea de transmisión.
- Las trazas que son mucho más largas que un cuarto de la longitud del borde más rápido comenzarán a comportarse como líneas de transmisión.

7. Distribución de reloj:

- La distribución de las señales de reloj debe realizarse de una manera que comprometa la necesidad de minimizar la desviación del reloj
- También se deben evitar los reflejos que pueden causar transiciones de reloj inaceptables debido a los efectos de la línea de transmisión.
- Las señales de reloj también deben aislarse de otras señales para evitar la diafonía entre el reloj y otras señales.
- Por lo general, las señales de reloj NO deben bloquearse para evitar efectos secundarios indeseables.

8. Distribución de energía y tierra:

- Se recomiendan tener planos de tierra y de potencia en circuitos impresos siempre que sea posible

- Permiten conexiones de baja impedancia y proporcionan un desacoplamiento de alta frecuencia de la capacitancia entre planos.
- Las conexiones a tierra deben ser lo más cortas posible, especialmente para los pines de tierra en varios dispositivos lógicos de salida, para evitar el rebote de tierra.

9. Entradas asincrónicas:

- Las entradas asíncronas deben sincronizarse utilizando dos niveles de flip-flops para minimizar la probabilidad de un estado metaestable cuando se muestrean las entradas asíncronas.

10. Estado de reinicio de encendido garantizado:

- Verifique que cualquier dispositivo se restablezca a un estado conocido cuando se aplica energía o cuando la energía cae por debajo de los niveles operativos normales (condición de caída de tensión).
- CPU, contadores, registros y dispositivos de memoria están sujetos a un comportamiento impredecible cuando la energía está fuera de las especificaciones y deben reiniciarse cuando la energía regrese a niveles especificados.

11. Problemas de compatibilidad electromagnética:

- Las señales que entran y salen de las placas de circuito impreso deben filtrarse para reducir al máximo la emisión involuntaria de radiofrecuencias.
- Los circuitos digitales también deberían empaquetarse en cajas conductoras para minimizar la radiación de las señales como interferencia a otros dispositivos y para proteger el dispositivo de interferencias externas y descargas.

Los relojes también deben mantenerse alejados de las señales de E / S y los conectores para reducir el acoplamiento del ruido del reloj a los cables e interconexiones que pueden actuar como antenas (Harold, 2001).



Tema 2.2: Arquitectura de hardware embebido

2.2.1 Arquitectura de Microcontroladores

Recordemos que un microcontrolador incorpora en el chip del microprocesador varios componentes periféricos comunes en el control de aplicaciones, tales como: periféricos de comunicación serial, temporizadores, contadores, circuito oscilador, puertos de entrada/salida (E/S), entre otros. También se los ha llamado microcomputadores.

Debido a su muy bajo costo, alta inmunidad al ruido eléctrico y pequeño tamaño, produjeron la revolución microcontrolada: Desplazó a toda la lógica cableada (utilizada en la electrónica industrial) y a la lógica programada (realizada con microprocesadores). Debido a que son la parte central del hardware de un sistema embebido, estudiaremos las arquitecturas de los microcontroladores existentes.

La arquitectura de un microcontrolador se refiere principalmente a la estructura de funcionamiento o composición del mismo. Desde este punto de vista general, se puede especificar la arquitectura de un microcontrolador desde enfoques más concretos. Veremos algunos de estos enfoques.

Según la conexión de componentes.

Atendiendo a la forma en que están conectados los bloques de componentes internos de un MCU (principalmente CPU y memoria), existen dos arquitecturas:

- **Von Newman:** Emplea un bus común para todos los componentes del microcontrolador. Aquí, los datos y las instrucciones circulan por el mismo bus, ya que ambos se guardan en la misma memoria (Figura 4). Esta arquitectura es más simple, pero el rendimiento no es tan alto, debido a que se pueden generar cuellos de botella durante la ejecución del código.
- **Harvard:** Esta arquitectura conecta la CPU hacia la memoria mediante dos buses distintos, uno de datos y otro de instrucciones (Figura 5). Aquí se obtiene mejores prestaciones (mayor rendimiento), pero en cambio el MCU puede tener mayor complejidad en su diseño y construcción (Benchimol, 2011).

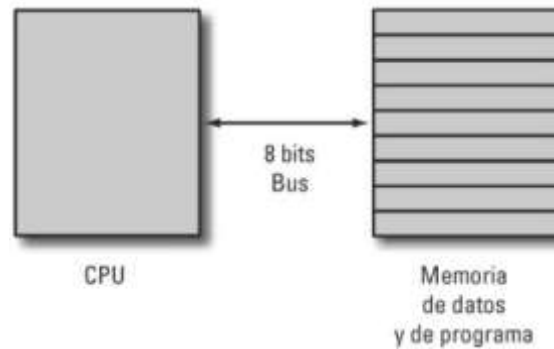


Figura 4. Esquema de Arquitectura Von Newman (Benchimol, 2011).

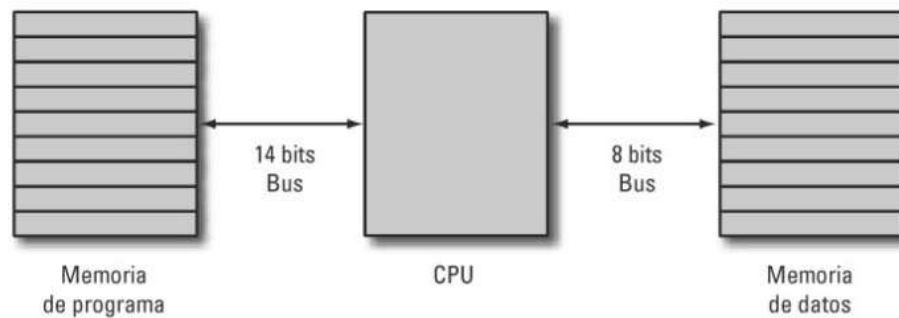


Figura 5. Esquema de Arquitectura Harvard (Benchimol, 2011).

Según complejidad del conjunto de instrucciones.

Al diseñar y construir un MCU (y un sistema embebido en general), el fabricante debe decidir si optimiza el hardware o el software. De acuerdo a esto se puede obtener un conjunto de instrucciones con mayor o menor complejidad. Esto da lugar a las arquitecturas CISC y RISC que ya hemos visto antes:

- **CISC (Complex Instruction Set Computing):** Emplea un conjunto de instrucciones más amplio (palabras de instrucción de longitud variable). Aumenta la simplicidad de la codificación (programas más simples, pero también aumenta la complejidad del hardware. Poco común en microcontroladores.
- **RISC (Reduced Instruction Set Computing):** Diseñada a un enfoque en instrucciones simples. Simplifica la estructura del hardware pero da como resultado programas más largos. Es la arquitectura más usual en microcontroladores (Jiménez, Palomera, & Couvertier, 2014).

Según el tamaño de palabra (ancho de bus).

Este criterio se refiere al ancho de bits de los buses internos del microcontrolador. Aunque este aspecto se refiere más a la capacidad del MCU que de su estructura, algunos autores lo toman como un enfoque de arquitectura.

Define tanto el tamaño máximo de valores que puede procesar, así como la cantidad máxima de instrucciones que se puede disponer. A mayor cantidad de bits, más posibles instrucciones y mayor cantidad de memoria que se puede gestionar. Por ejemplo: Un MCU de 8 bits solo podrá manejar como máximo 2^8 direcciones de memoria (256), si cada espacio de memoria fuera de 1byte (considerando que es el tamaño del bus, o bits), esto daría como resultado una memoria de 256B.

De acuerdo a este criterio, los microcontroladores pueden tener arquitecturas de:

- 4-bit, 12-bit (muy raros), 16-bit, 32-bit, y 64-bit.

Según especificaciones de diseño o familia.

Se refiere al diseño interno específico de un conjunto de microcontroladores muy similares, generalmente de un mismo fabricante, y que solo difieren en detalles como la cantidad de memoria interna, velocidad de reloj, o cantidad de pines de E/S. Estas especificaciones dependen de la empresa que haya diseñado o fabricado el microcontrolador. En algunos casos, este conjunto de MCUs similares puede constituirse una **familia** de microcontroladores.

Se pueden encontrar varias arquitecturas de diseños específicos de microcontrolador, pero mencionaremos solo algunas que se han destacado:

- **Motorola serie 68000:** También conocida como 680x0 , m68000 , m68k o 68k) es una familia de cpu y mcu de tipo CISC de 32 bits, aunque también hubieron versiones de 8 y 16 bits. Utilizados también como CPU de algunas PC y consolas de videojuegos de los 80s e inicio de los 90s. Tiene ocho registros de datos de uso general de 32 bits (D0-D7) y ocho registros de direcciones (A0-A7).
- **Intel 8051:** Comúnmente denominado solo 8051, es una serie de microcontroladores de un solo chip desarrollada por Intel en 1980 para su uso en sistemas embebidos. Es de tipo CISC, aunque tiene características de tipo RISC. Las versiones originales de Intel fueron populares en la década de 1980 y

principios de la de 1990 y los derivados compatibles con binarios mejorados siguen siendo populares en la actualidad, aunque ya no los fabrica Intel.

- **PIC:** Es una familia de microcontroladores fabricados por Microchip Technology, derivados del PIC1650. Existieron versiones disponibles en 12, 14, 16, 18 y 24 bits. Las capacidades de hardware de los dispositivos PIC van desde SMD de 6 pines, chips DIP de 8 pines hasta chips SMD de 144 pines, con pines de E/S discretos, módulos ADC y DAC y puertos de comunicaciones como UART, I2C, CAN, e incluso USB.
- **AVR:** Es una familia de microcontroladores desarrollados desde 1996 por Atmel, (adquiridos por Microchip Technology en 2016). Son MCUs de un solo chip RISC de 8 bits con arquitectura Harvard modificada. Fue una de las primeras familias de microcontroladores en usar memoria flash en chip para el almacenamiento de programas, a diferencia de otros que usaban ROM, EPROM o EEPROM. Los microcontroladores AVR encuentran muchas aplicaciones de sistemas embebidos. Son especialmente comunes en aplicaciones integradas educativas y de aficionados, popularizadas por su inclusión en muchas de las placas de desarrollo de la línea Arduino.
- **ARM:** Incluye un conjunto de diseño de microcontroladores. La Advanced RISC Machines (originalmente Acorn RISC Machine) es una empresa que diseñó una familia de arquitecturas informáticas de conjunto de instrucciones reducidas (RISC) para procesadores de computadora y MCU, para varios entornos. Ha habido varias generaciones y conjuntos de diseño ARM, que van desde los 24 a 64 bits. Dentro de los diseños de ARM hay arquitecturas definidas:
 - ARM1-ARM6, Cortex M, Cortex R, Cortex A



Dato útil: En esta página de Wikipedia en inglés hay una lista amplia de familias de microcontroladores agrupadas por fabricante, y dentro de cada fabricante agrupados por el tamaño de palabra o bus en bits. Es muy posible que la lista no muestre todos los fabricantes ni todas las familias, pero es una referencia bastante buena: https://en.wikipedia.org/wiki/List_of_common_microcontrollers .

2.2.2 Ejemplos de Hardware: Interfaces GPIO y seriales

Ejemplos de microcontroladores.

En clases y prácticas previas hemos visto algunos microcontroladores específicos. Incluso los hemos clasificado de acuerdo a su arquitectura bajo distintos criterios (CISC y RISC, tamaño de palabra en bits, especificaciones de diseño...). Si bien se puede adquirir el chip de un microcontrolador de forma suelta, es muy común encontrarlos (y comprarlos) como parte de una **placa de evaluación**. Una placa o *board* de evaluación es una placa de circuito integrado que trae incorporado un microcontrolador, otros chips complementarios (reguladores de corriente, depuración, conversor serial UART-USB, entre otros), así como pines (macho o hembra) para conectar otros componentes. Como su nombre lo dice, la finalidad de esta placa es probar diseños o prototipos que utilicen el microcontrolador incluido, pero también pueden utilizarse para construir un dispositivo funcional.

Como ejemplos de estas placas tenemos:

- **Arduino UNO:** Una de las plataformas de hardware abierto más conocidas, utiliza un MCU Microchip ATmega328P de 8-bit con 23 pines, 2KB de RAM, 1KB de EEPROM, 32KB de memoria flash, y una velocidad de CPU de 20MHz.
- **Raspberry Pi 4 modelo B:** Aunque el objetivo de esta placa es proveer un computador barato, se utiliza mucho en sistemas embebidos. Esta versión tiene un SoC (*System-on-Chip*) con un procesador ARM-Cortex A72 de 64-bit a 1.5GHz, 2GB de RAM, y varias opciones de comunicación (Ethernet, WiFi, Bluetooth).
- **NodeMCU:** Es una placa de bajo costo, orientada al Internet de las Cosas (IoT), que cuenta con un microcontrolador Espressif ESP8266. Este MCU tiene una CPU de 32-bit a 160MHz, 96KB de RAM, 4MB de memoria flash, y 17 pines para conectar componentes.

En la Figura 6 se muestran algunas placas de desarrollo o evaluación, junto con las características de sus microcontroladores.

Criteria	Analyzed PCB				
	Arduino Uno R3	Arduino Mega 2560 R3	NodeMcu V3	Yubox v1.0	Nucleo64-STM32 L452
MCU	ATmega 328P	ATmega 2560	Tensilica Xtensa LX3 ESP8266	ESP32	STM32L452RE
Clock Speed	20 MHz	16 MHz	160MHz	240MHz	80 MHz
Flash memory	32K	256K	4MB	4 MB	512 KB
RAM	2KB	8KB	96KB	520 KB	160KB
Digital pins	14	54	17	12	51
SD/SDIO	No	No	Si	Si	Si
I2C	1	1	1	2	4^a
SPI	1	1	1	1	3^a
UART	1	4	1	2	4^a
Operation conditions	-40 ~ +85 °C	-40 ~ +85°C	-40 ~ +85°C	-40 ~ +85°C	-40 ~ +130 °C

Figura 6. Tabla comparativa de varias placas de desarrollo ((Navia, Palma, Moreira, & Velez-Valarezo, 2022)).

De estos microcontroladores y placas de desarrollo disponibles, en esta asignatura se utilizará el NodeMCU para las prácticas. Se seleccionó esta placa debido a su bajo costo y su capacidad de conectarse a Internet a través de una red WiFi. Cuenta con un puerto microUSB de alimentación y comunicación con un PC para poder programar o depurar su funcionamiento, para lo que dispone también de un convertidor TTL-USB. En la Figura 7 se muestra una fotografía con el aspecto y los elementos que incluye esta placa, mientras que la Figura 8 muestra la función de los pines.

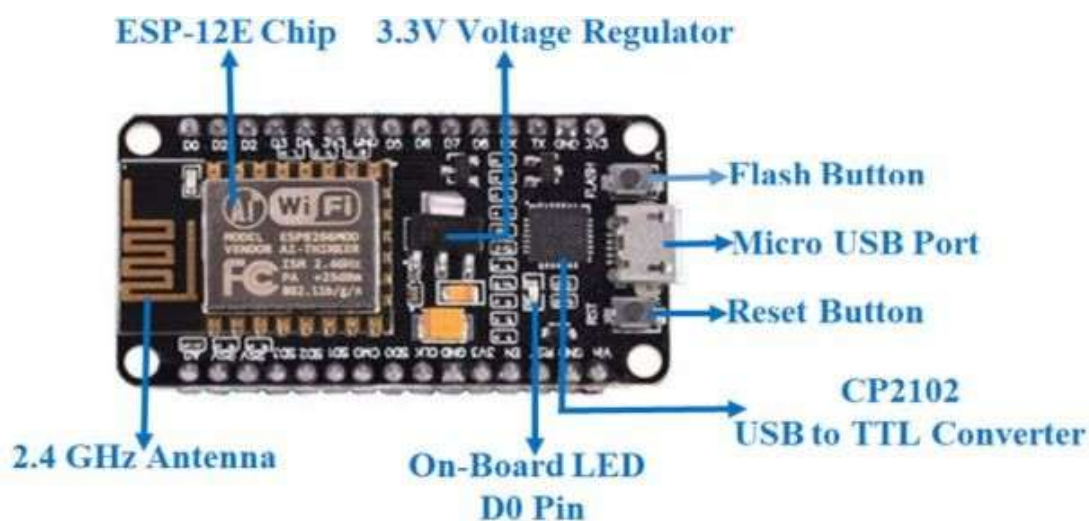


Figura 7. Elementos de la placa NodeMCU.

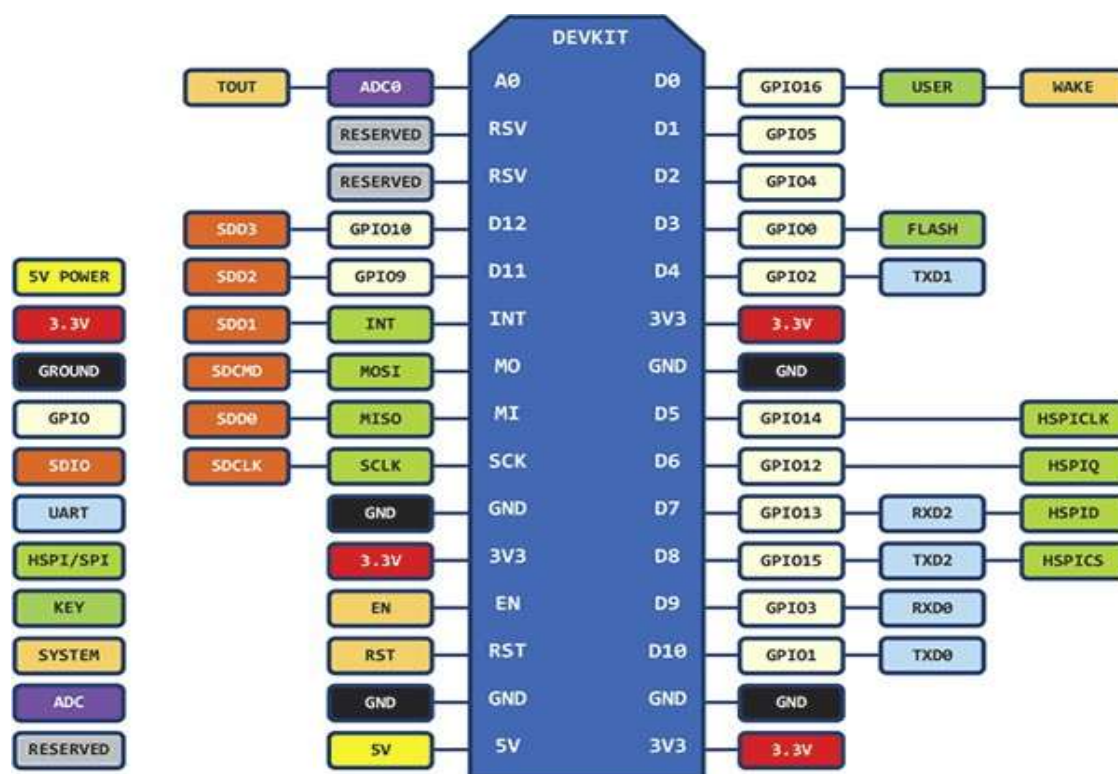


Figura 8. Diagrama de pines del NodeMCU.

Entornos de programación de hardware embebido.

Como vimos en la Unidad 1, la programación del hardware embebido puede realizarse utilizando distintos lenguajes de programación. Para realizar esto, se debe tener un conjunto de programas, o un entorno de desarrollo adecuado. En la siguiente unidad se estudiará mejor el proceso de compilación del código y carga del programa en el hardware embebido. Por ahora nos centraremos en el uso de un IDE para programar nuestro hardware.

El IDE que utilizemos depende principalmente de la placa que se utilice para los dispositivos embebidos, aunque también podría depender del lenguaje que queramos utilizar. Existen opciones en línea, pero principalmente para sistemas operativos de escritorio. Algunas opciones de IDEs para escritorio se muestran en la Figura 9.

Uno de los IDE más utilizados en la actualidad, y que es recomendado para la mayoría de proyectos de aprendizaje, es **Arduino IDE** (Figura 10). Se trata de un IDE abierto y gratuito. Aunque fue desarrollado en principio para las placas de la marca Arduino, soporta programación para otras placas/plataformas, que se pueden agregar mediante librerías o paquetes.



Figura 9. Logotipos de varios IDEs para programar microcontroladores.

El Lenguaje de Arduino IDE está basado en C/C++, pero algo simplificado. Por lo tanto, los tipos de variables, bucles, y palabras reservadas utilizadas en el código con las mismas. Este IDE se puede descargar en <https://www.arduino.cc/en/software>.



Figura 10. Interfaz de Arduino IDE.

Comunicación en paralelo y serie: GPIO.

La comunicación del microcontrolador con componentes externos (sensores o actuadores) se da por medio de interfaces que pueden ser Analógicas o Digitales. Las interfaces **analógicas** requieren de un convertidor que permita su procesamiento digital. Estos convertidores pueden ser:

- *Analog-Digital Converter* (ADC), que se utiliza para transformar los datos analógicos de entrada en un equivalente digital que pueda ser procesado por el microcontrolador.
- *Digital-Analog Converter* (DAC), empleado para generar una señal analógica de salida con base en un valor digital.

Las interfaces digitales permiten transmitir en paralelo o en serie. La comunicación en paralelo permite enviar varios bits en una sola señal de reloj, pero requiere más líneas de comunicación; mientras que la comunicación en serie requiere menos líneas, pero solo se envía un bit por cada ciclo (Figura 11). Para ambas opciones se tiene:

- En paralelo: GPIO
- En serie: UART / RS-232, SPI, I2C.

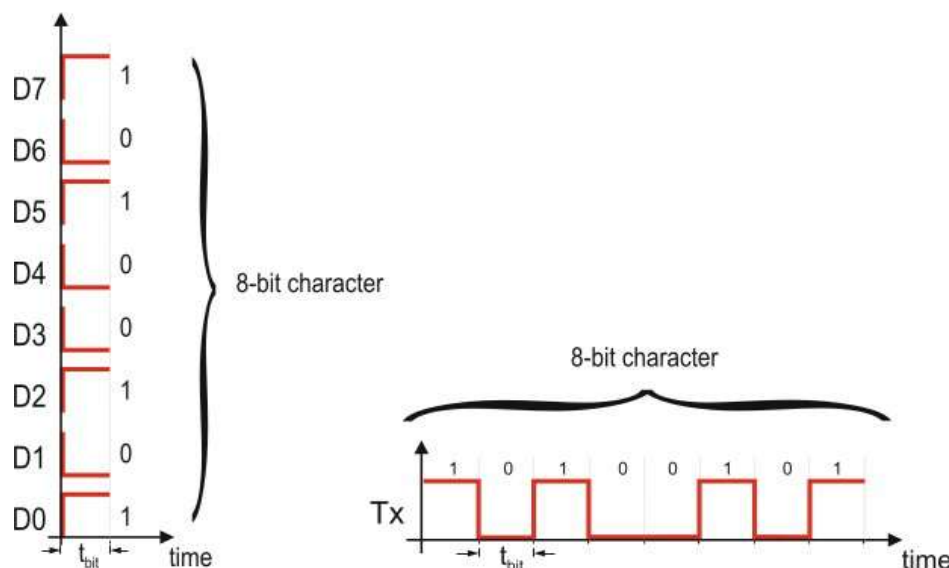


Figura 11. Comunicación en paralelo y en serie

Los **GPIO** (*General Purpose Input-Output*) son puertos de E/S que permiten enviar o recibir una señal digital al MCU. Un puerto GPIO consta de un grupo de líneas de E/S que pueden configurarse, de forma independiente o en grupo, como entradas o como

salidas. La mayoría de las MCU asocian 8 líneas de E/S a un puerto, aunque es posible encontrar puertos de 4, 6, o 16 líneas.

Las líneas GPIO están disponibles para el exterior a través de pines de paquete dedicados o compartidos. La nomenclatura con la que se denomina un puerto GPIO y sus pines depende del fabricante. Por ejemplo

- P0 (P0.1, P0.2...P0.8) , P1, P2...
- PA (PA.0, PA.1...PA.7) , PB, PC...
- GPIOA (GPIOA.1, GPIOA.2...GPIOA.8), GPIOB, GPIOC...

Los puertos GPIO pueden funcionar como de entrada o salida. Para esto es necesario configurarlos, mediante software, antes de su uso. La Figura 12 muestra el esquema electrónico de un pin de un puerto GPIO.

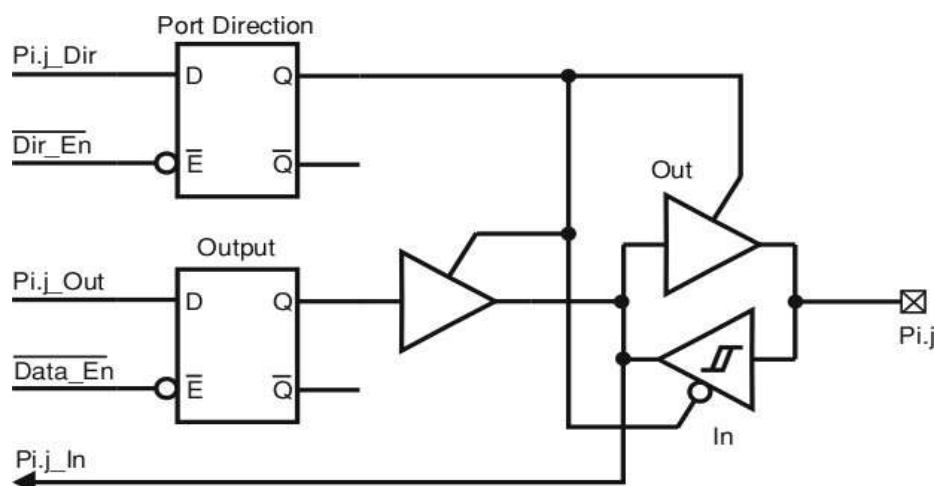


Figura 12. Esquema electrónico de un pin GPIO (Jiménez, Palomera, & Couvertier, 2014).

Interfaces seriales de comunicación.

Una interfaz serie que funciona como transmisor o controlador envía bits de uno en uno a una entrada de puerto serie que funciona como receptor; normalmente en un componente diferente.

El cable entre los componentes generalmente tiene una ruta de datos dedicada para cada dirección. Algunas interfaces seriales tienen una única ruta de datos compartida para ambas direcciones, y los transmisores se turnan. En otros casos puede haber una línea de transmisión (Tx) y una de recepción (Rx). En este sentido, la comunicación serial puede ser simplex, half-duplex, o full-duplex (Figura 13).

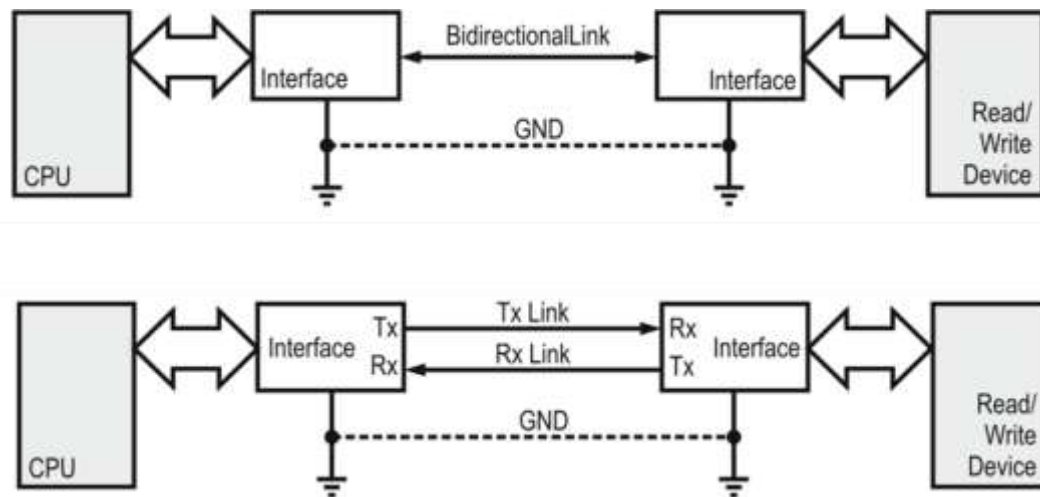


Figura 13. Comunicación half-duplex y full-duplex (Jiménez, Palomera, & Couvertier, 2014)).

Además, la comunicación serial requiere algún tipo de sincronización para saber la frecuencia de envío de los bits. En este sentido la comunicación puede ser:

- Síncrona: Cuando además de la línea de datos existe una línea con una señal de reloj, que indica la duración de cada bit. No es necesario configurar la velocidad de transmisión en el receptor. Solo se transmite mientras la señal de reloj esté activa.
- Asíncrona: No hay un reloj de referencia, sino que en ambos elementos (transmisor y receptor) se configura la velocidad de transmisión de los datos. Aquí por lo general se envía primero una señal de inicio de transmisión, seguido los datos, y al final una señal de parada (Alexon, 2007).

La Figura 14 muestra ambos modos de transmisión.

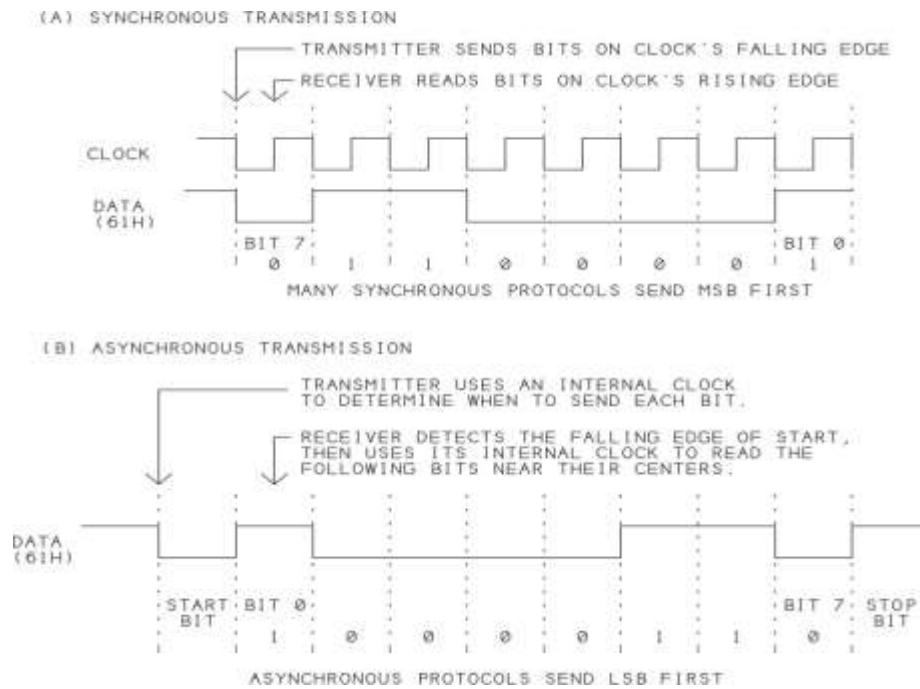


Figura 14. Transmisión Síncrona y Asíncrona (Alexon, 2007).

Existen varios protocolos de comunicación serial (como por ejemplo el estándar de red Ethernet). En esta sección veremos los más utilizados en sistemas embebidos:

- **UART (*Universal Asynchronous Receiver and Transmitter*):** Se denomina UART a cualquier módulo de interfaz para un canal de comunicación en serie asíncrono. Cuando se usa una UART es necesario acordar (configurar) la velocidad en baudios, el número de bits de datos por carácter, si se utilizará o no la paridad y el número de bits de parada que se utilizarán al comunicarse. Dependiendo del microcontrolador, del lenguaje de programación, y del entorno de desarrollo; este proceso puede variar un poco. La configuración de la UART requiere por lo general los siguientes pasos:
 - Elección de la fuente de reloj para el (los) generador (es) de velocidad en baudios (bps).
 - Configuración del generador de velocidad en baudios.
 - Elección del modo de sincronización correcto (si soportara ambos modos)
 - Elección y configuración de la comprobación de paridad
 - Configuración de la longitud y bits de parada de carácter
 - Habilitación del puerto UART (Jiménez, Palomera, & Couvertier, 2014).

- SPI (Serial Peripheral Interface):** Es un estándar de bus serie síncrono con capacidad de full-duplex creado por Motorola para admitir comunicaciones entre un host maestro y uno o varios dispositivos periféricos esclavos. SPI es uno de los protocolos de comunicaciones síncronas más simples: solo establece un mecanismo básico para retransmitir paquetes entre un maestro y uno o más esclavos, sin especificar dato, sesión o protocolo de nivel superior. Es protocolo con poca sobrecarga. Un controlador SPI se desarrolla alrededor de un registro de desplazamiento único que sirve como receptor y transmisor, sincronizado por el reloj de transmisión. El canal presenta cuatro señales cuyas funciones son:
 - SCLK: Reloj serial, enviado por el maestro y sincronizando maestro y esclavo.
 - SDO: salida de datos en serie, el flujo de salida en serie del dispositivo.
 - SDI: entrada de datos en serie, el flujo de entrada en serie en el dispositivo.
 - SS: Selección de esclavo, línea de selección para habilitar el esclavo. Se omite en las conexiones punto a punto (Figura 15).

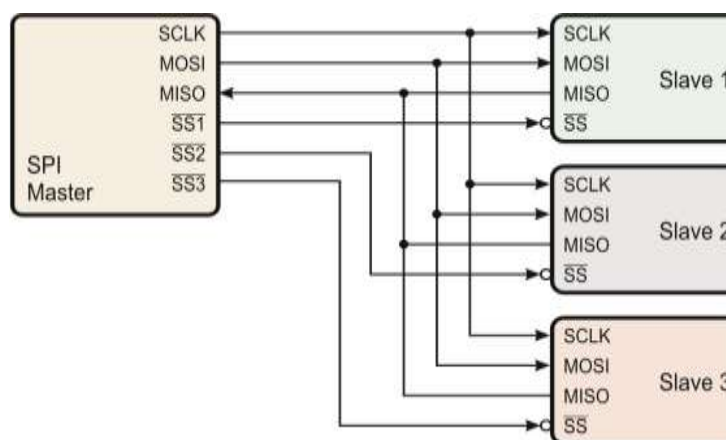


Figura 15. Esquema de comunicación por SPI.

- I²C (Inter-Integrated Circuit Bus):** Es un protocolo serial síncrono desarrollado por Philips Semiconductor (ahora NXP Semiconductors) a principios de la década de 1980. Admite la interconexión a nivel de placa de módulos IC y periféricos. El protocolo utiliza dos líneas de comunicación: SDA (datos en serie) y SCL (reloj en serie), (además de tierra) para establecer un bus multipunto, maestro-esclavo, half-duplex, capaz de manejar múltiples maestros y esclavos. La línea de reloj en serie (SCL) sincroniza todas las transferencias de bus, mientras que SDA transporta los datos que se transfieren (Figura 16).

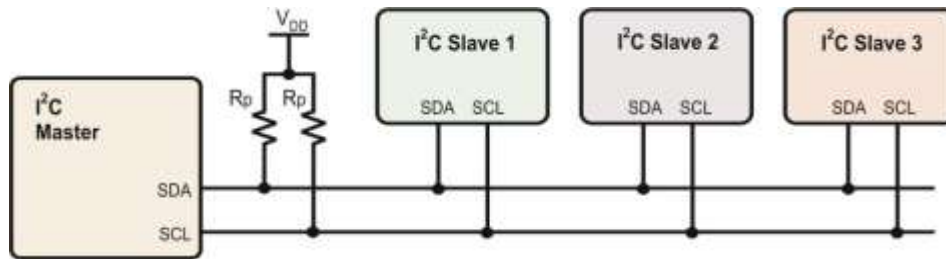


Figura 16. Esquema de comunicación por I2C



Recuerda que: Existen otros estándares de comunicación en serie, como el USB que veremos más adelante.



Video: En este video puedes revisar una explicación sobre el funcionamiento de la comunicación serial <https://www.youtube.com/watch?v=G7aQB6x0LHc&t=112s>.

Tema 2.3: Diseño lógico de hardware embebido

2.3.1 Diseño con lenguajes de descripción de hardware

Los Lenguajes de Descripción de Hardware digital, **HDL** por sus siglas en inglés (*Hardware Description Languages*) son lenguajes que describen el hardware de los sistemas digitales en forma textual. Se parecen a los lenguajes de programación, pero están orientados específicamente a la descripción de las estructuras y el comportamiento del hardware.

Hay muchos HDL exclusivos en la industria, desarrollados por empresas que diseñan o ayudan a diseñar circuitos integrados. El IEEE (Instituto de Ingenieros en Electricidad y Electrónica) apoya dos HDL estándar: **VHDL** y **Verilog HDL**.

VHDL es un lenguaje cuyo uso exige el Departamento de Defensa de EU y que inicialmente usaban los contratistas de esa dependencia, aunque ahora se le usa comercialmente y en universidades de investigación (Morris Mano, 2003).

VHDL proporciona tres métodos básicos para describir un circuito digital por software: **comportamental**, **flujo de datos** y **estructural**. Aunque no es el objetivo de esta asignatura explicar a detalle en que consiste cada uno de ellos, podemos mencionar la forma básica en la que se estructura VHDL:

- Una “entidad” (*entity*) es la abstracción de un circuito, ya sea desde un complejo sistema electrónico o una simple puerta lógica. Solo define la forma externa del circuito (entradas y salidas), no su funcionamiento.
- La “arquitectura” (*architecture*) describe el funcionamiento de la entidad a la que hace referencia, es decir, describe el funcionamiento de la entidad a la que está asociada utilizando las sentencias y expresiones propias de VHDL.
- Un “proceso” (*process*) es una estructura particular de VHDL que se reserva principalmente para contener sentencias que no tengan obligatoriamente que tener definido su valor para todas las entradas (Figura 17) (Sánchez-Élez, 2014).

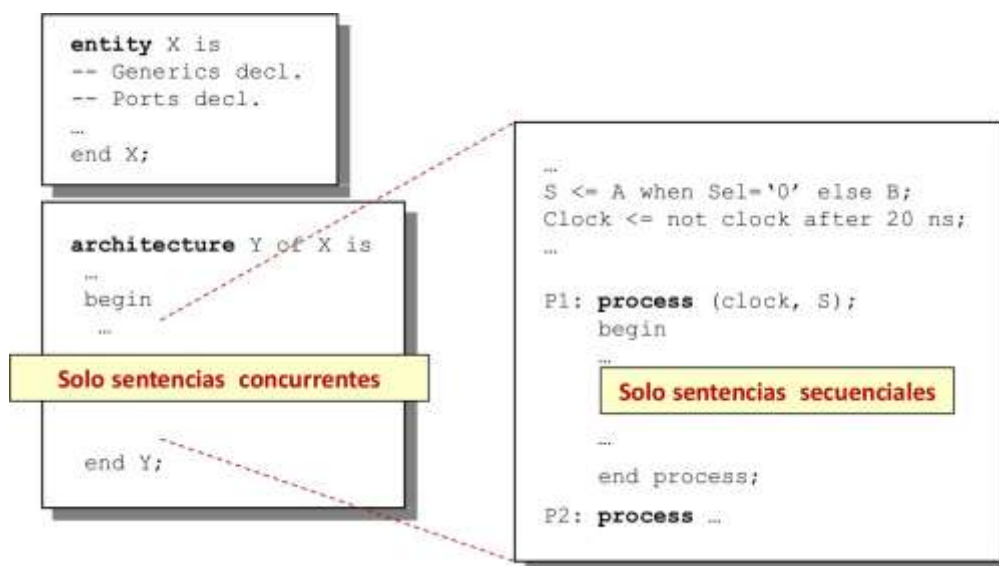


Figura 17. Estructura general de un modelo en VHDL

El código de VHDL se guarda en archivos con extensión *.vhd*. VHDL, así como otros lenguajes HDL, es una herramienta que permite implementar diseños digitales y es, por tanto, un medio para conseguir un fin y no un fin en sí mismo (Floyd, 2006).



Sabías que: Existen también otros HDL como C/C++/SystemC/SystemVerilog, con mayor nivel de abstracción para especificar sistemas Hw/Sw a nivel funcional.

2.3.2 Descripción estructural, de flujo de datos y de comportamiento.

El lenguaje VHDL es una herramienta muy poderosa para el diseño de sistemas electrónicos. Se puede aplicar para el diseño de un sistema embebido, especialmente en el diseño del microcontrolador de dicho sistema, pero también para el diseño del funcionamiento de otras partes del mismo.

En esta sección vamos a analizar un ejemplo de este tipo de definición, es decir, se analizará el código de definición (de estructura y funcionamiento) de un microcontrolador genérico. La finalidad de este diseño en VHDL es netamente didáctica, y no se profundizará demasiado en el análisis de su funcionamiento.

Microcontrolador a diseñar.

Se diseñará un microcontrolador (computador-en-chip) genérico de 8 bits. Este microcontrolador tiene:

- Un procesador (CPU) de 8 bits
- Una memoria de programa de 128 bytes
- Una RAM de 96 bytes
- Puertos de salida de 16x8 bits y puertos de entrada de 16x8 bits.

Los usuarios pueden programar el microcontrolador insertando códigos de operación y operandos en la memoria del programa.

Este diseño está basado en el diseño presentado por (LaMeres, 2017), e implementado con VHDL en (FPGA4student, 2017). El esquema de los componentes de este microcomputador se puede apreciar en la Figura 18. Se definen tanto las entradas y salidas del sistema completo, así como la comunicación entre los componentes internos.



Dato útil: Los archivos completos de este diseño de microcontrolador están disponibles en <https://www.fpga4student.com/2016/12/a-complete-8-bit-microcontroller-in-vhdl.html>.



Video: En este video puedes ver otra explicación sobre que es VHDL y sus diferencias con los lenguajes de programación de alto nivel conocidos <https://www.youtube.com/watch?v=XFyu9JhG7Nw>.

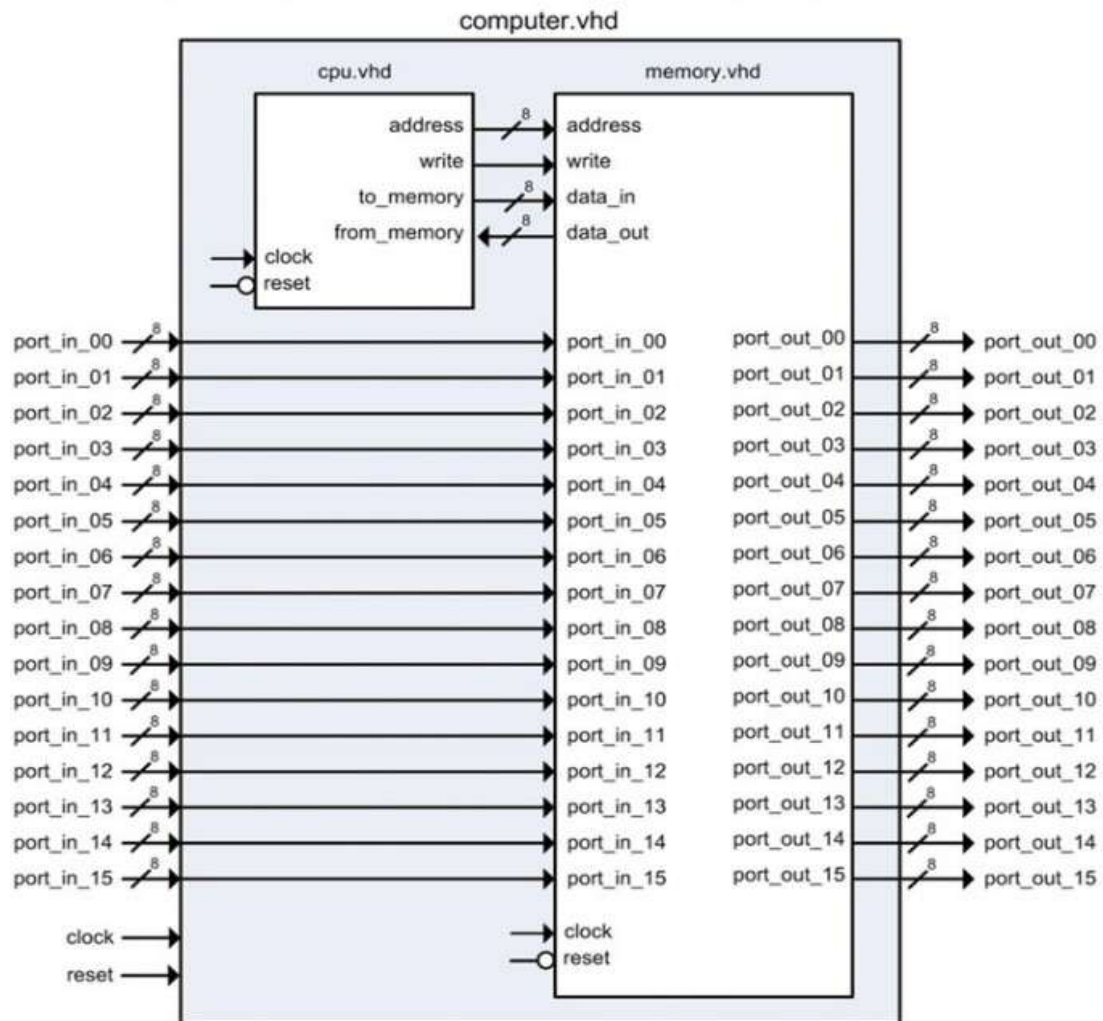


Figura 18. Diagrama de bloque del microcomputador de 8-bits (LaMeres, 2017)

También se muestra un conjunto de instrucciones básicas para nuestro sistema de ejemplo (Figura 19). Este conjunto proporciona una variedad de cargas (loads) y almacenamientos, manipulaciones de datos e instrucciones de rama (branch) que permitirán programar la computadora para realizar tareas más complejas a través del desarrollo de software. Estas instrucciones son suficientes para proporcionar una base de funcionalidad para que el sistema embebido esté operativo. Se pueden agregar instrucciones adicionales según se desee para aumentar la complejidad del sistema

Mnemonic	Opcode, Operand	Description
"Loads and Stores"		
LDA_IMM	x"86", <data>	Load Register A using Immediate Addressing
LDA_DIR	x"87", <addr>	Load Register A using Direct Addressing
LDB_IMM	x"88", <data>	Load Register B with Immediate Addressing
LDB_DIR	x"89", <addr>	Load Register B with Direct Addressing
STA_DIR	x"96", <addr>	Store Register A to Memory using Direct Addressing
STB_DIR	x"97", <addr>	Store Register B to Memory using Direct Addressing
"Data Manipulations"		
ADD_AB	x"42"	A = A + B (plus)
SUB_AB	x"43"	A = A - B (minus)
AND_AB	x"44"	A = A · B (AND)
OR_AB	x"45"	A = A + B (OR)
INCA	x"46"	A = A + 1 (plus)
INCB	x"47"	B = B + 1 (plus)
DECA	x"48"	A = A - 1 (minus)
DECB	x"49"	B = B - 1 (minus)
"Branches"		
BRA	x"20", <addr>	Branch Always to Address Provided
BMI	x"21", <addr>	Branch to Address Provided if N=1
BPL	x"22", <addr>	Branch to Address Provided if N=0
BEQ	x"23", <addr>	Branch to Address Provided if Z=1
BNE	x"24", <addr>	Branch to Address Provided if Z=0
BVS	x"25", <addr>	Branch to Address Provided if V=1
BVC	x"26", <addr>	Branch to Address Provided if V=0
BCS	x"27", <addr>	Branch to Address Provided if C=1
BCC	x"28", <addr>	Branch to Address Provided if C=0

Figura 19. Ejemplo de conjunto de instrucciones para el MCU de 8-bits (LaMeres, 2017).

Implementación de Memoria de Programa

La Figura 20 muestra el diagrama de bloques del sistema de memoria. Este sistema está compuesto por varios componentes de más bajo nivel, definidos en los archivos *rom_128x8_sync.vhd* y *rw_96x8_sync.vhd*. Un multiplexor es utilizado para enrutar los datos de salida.

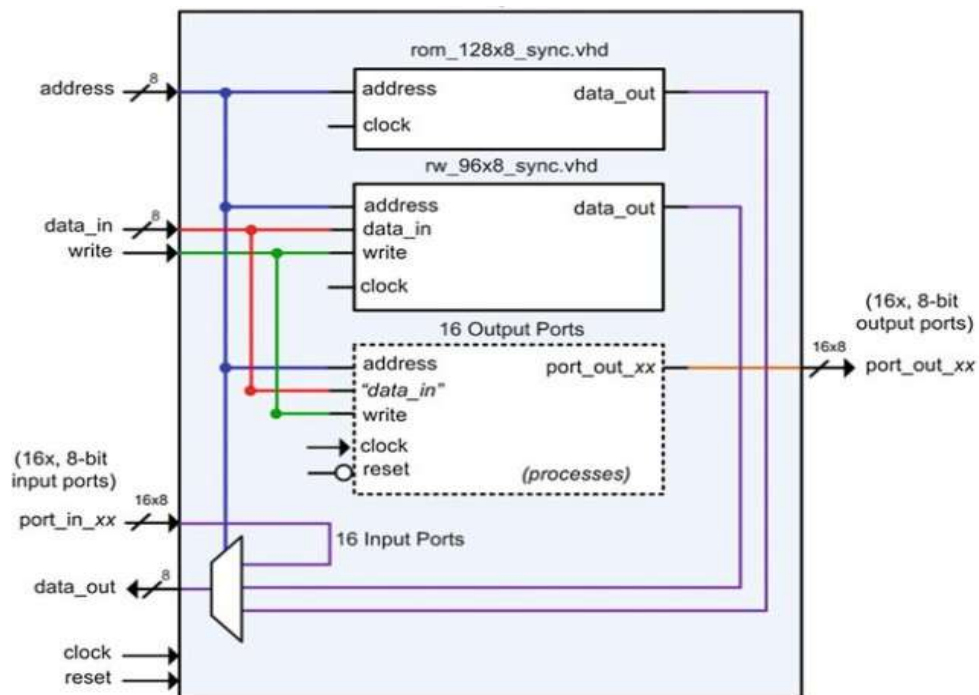


Figura 20. Diagrama de implementación de memoria (LaMeres, 2017).

Para que el VHDL sea más legible, las instrucciones se pueden declarar como constantes. Esto permite usar un mnemotécnico (un alias) cuando se llene la matriz de memoria de programa. El siguiente código VHDL muestra cómo se pueden definir como constantes los mnemotécnicos para el conjunto de instrucciones:

```
constant LDA_IMM : std_logic_vector (7 downto 0) := x"86";
constant LDA_DIR : std_logic_vector (7 downto 0) := x"87";
constant LDB_IMM : std_logic_vector (7 downto 0) := x"88";
constant LDB_DIR : std_logic_vector (7 downto 0) := x"89";
constant STA_DIR : std_logic_vector (7 downto 0) := x"96";
constant STB_DIR : std_logic_vector (7 downto 0) := x"97";
constant ADD_AB  : std_logic_vector (7 downto 0) := x"42";
constant SUB_AB  : std_logic_vector (7 downto 0) := x"43";
constant AND_AB  : std_logic_vector (7 downto 0) := x"44";
constant OR_AB   : std_logic_vector (7 downto 0) := x"45";
constant INCA    : std_logic_vector (7 downto 0) := x"46";
constant INCB    : std_logic_vector (7 downto 0) := x"47";
constant DECA    : std_logic_vector (7 downto 0) := x"48";
constant DECB    : std_logic_vector (7 downto 0) := x"49";
...
```

Ahora el programa puede ser declarado como un tipo *Array* con valores iniciales. El siguiente código VHDL muestra como declarar el programa de memoria, y un ejemplo de código para ejecutar siempre un *load* (cargar), *store* (almacenar), y *branch* (bifurcación). El programa básicamente escribe continuamente *x"AA"* en *port_out_00*:

```
type rom_type is array (0 to 127) of std_logic_vector(7 downto 0);

constant ROM : rom_type := (0      => LDA_IMM,
                             1      => x"AA",
                             2      => STA_DIR,
                             3      => x"E0",
                             4      => BRA,
                             5      => x"00",
                             others => x"00");
```

La asignación de direcciones para la memoria del programa se maneja de dos formas. Primero, se define un array con índices de 0 a 127, para proveer direcciones apropiadas para cada ubicación en la memoria. Segundo, se crear una línea de habilitación interna que solo permitirá asignaciones de ROM a *data_out* con dirección válida. Considerar el siguiente código VHDL para crear una habilitación interna (EN) que solo será activado cuando la dirección esté dentro del rango válido de memoria de programa (0-127):


```
enable : process (address)
begin
  if ((to_integer(unsigned(address)) >= 0) and
      (to_integer(unsigned(address)) <= 127)) then
    EN <= '1';
  else
    EN <= '0';
  end if;
end process;
```

Si la señal de habilitación no se crea, la simulación y al síntesis fallará porque la asignación de *data_out* estará fuera del rango de direcciones válidas de memoria. Esta línea de habilitación puede ser usada en el modelo de comportamiento para el proceso de la ROM de esta manera:

```
memory : process (clock)
begin
  if (clock'event and clock='1') then
    if (EN='1') then
      data_out <= ROM(to_integer(unsigned(address)));
    end if;
  end if;
end process;
```

Implementación de Memoria de Datos

La memoria de datos es creada con similar estrategia que la de programa. Se define un array con un rango de direcciones correspondiente a las del sistema (en este caso 128-223). Un *habilitador* interno se crea para prevenir que *data_out* se asigne fuera de un rango válido de direcciones. El siguiente código declara el arreglo de memoria R/W:

```
type rw_type is array (128 to 223) of std_logic_vector(7 downto 0);
signal RW : rw_type;
```

El siguiente es el código VHDL para modelar la habilitación local y las asignaciones de señal para la memoria R/W:

```
enable : process (address)
begin
  if ((to_integer(unsigned(address)) >= 128) and
      (to_integer(unsigned(address)) <= 223)) then
    EN <= '1';
  else
    EN <= '0';
  end if;
end process;

memory : process (clock)
begin
  if (clock'event and clock='1') then
    if (EN='1' and write='1') then
      RW(to_integer(unsigned(address))) <= data_in;
    elsif (EN='1' and write='0') then
      data_out <= RW(to_integer(unsigned(address)));
    end if;
  end if;
end process;
```

Implementación de Puertos de E/S.

A cada puerto de salida en el sistema se le asigna una dirección única. Cada puerto de salida también contiene capacidad de almacenamiento. Esto permite que la CPU actualice un puerto de salida escribiendo en su dirección específica. Una vez que la CPU termina de almacenar en la dirección del puerto de salida y pasa a la siguiente instrucción en el programa, el puerto de salida retiene su información hasta que se vuelve a escribir. Este comportamiento se puede modelar mediante un proceso que utiliza el bus de direcciones y la señal de escritura para crear una condición de habilitación síncrona. El siguiente código VHDL muestra cómo se modelan los puertos de salida en `x"E0"` y `x"E1"` utilizando procesos específicos de dirección:

```
-- port_out_00 description : ADDRESS x"E0"
U3 : process (clock, reset)
begin
    if (reset = '0') then
        port_out_00 <= x"00";
    elsif (clock'event and clock='1') then
        if (address = x"E0" and write = '1') then
            port_out_00 <= data_in;
        end if;
    end if;
end process;

-- port_out_01 description : ADDRESS x"E1"
U4 : process (clock, reset)
begin
    if (reset = '0') then
        port_out_01 <= x"00";
    elsif (clock'event and clock='1') then
        if (address = x"E1" and write = '1') then
            port_out_01 <= data_in;
        end if;
    end if;
end process;

:
"the rest of the output port models go here..."
:
```

Los puertos de entrada no contienen almacenamiento, pero requieren un mecanismo que enrute selectivamente su información al puerto correcto de la memoria del sistema. Esto se puede lograr con un multiplexor.

2.3.3 Modelo, entorno y mecanismo de simulación.

La CPU contiene dos componentes, la unidad de control (*control_unit.vhd*) y la ruta de datos (*data_path.vhd*). La ruta de datos contiene todos los registros y la ALU. La ALU se implementa como un subcomponente dentro de la ruta de datos (*alu.vhd*). La ruta de datos también contiene un sistema de bus para facilitar el movimiento de datos entre los registros y la memoria. El sistema de bus se implementa con dos multiplexores que son controlados por la unidad de control. La unidad de control contiene la máquina de estados finitos que genera todas las señales de control para la ruta de datos mientras realiza los pasos de búsqueda, decodificación y ejecución de cada instrucción. La Figura 21 resume este esquema.

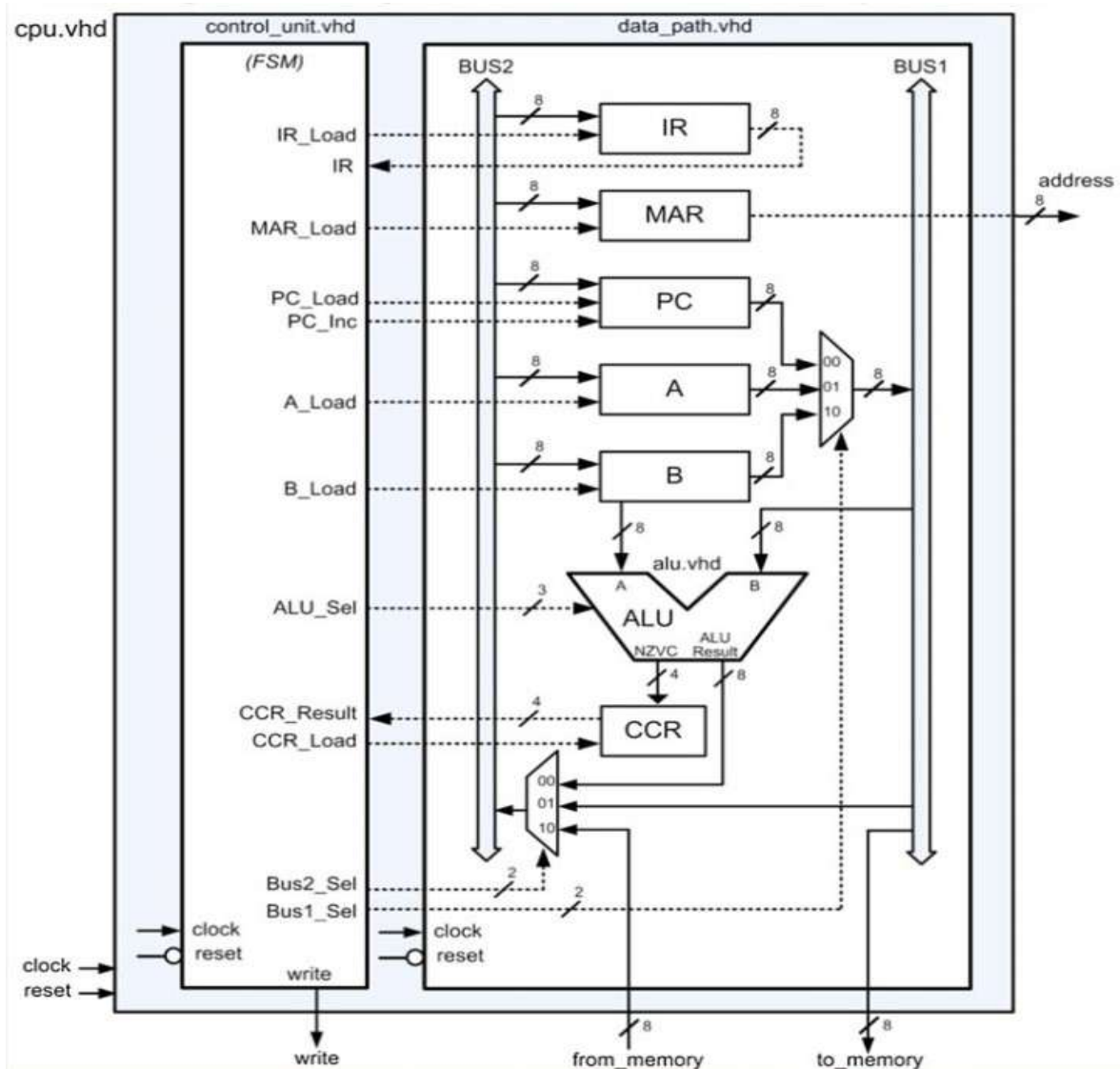


Figura 21. Diagrama de bloque del CPU para el computador de 8-bits (LaMeres, 2017).

Al igual que los elementos antes mencionados, los componentes de la CPU también se pueden definir con VHDL. En el texto de (LaMeres, 2017) se puede obtener la explicación de cada componente, mientras que en la web de (FPGA4student, 2017) se puede obtener todo el código en VHDL del microcontrolador de 8 bits. Dentro de este sitio web también está a disposición el código para simular el sistema diseñado.

Para efectuar la simulación se la puede hacer de dos maneras:

- Utilizando una FPGA (*Field-Programmable Gate Array*) que es un dispositivo programable que permite probar las funcionalidades de un código de HDL.
- Utilizando un simulador de VHDL, que es una aplicación para verificar, mediante software, el funcionamiento de un código en VHDL.

Esta última opción es la más utilizada, cuando se quiere hacer una primera evaluación del código. Existen varios simuladores en el mercado, tanto gratuitos como pagados. Uno de ellos, que está disponible en línea, es EDA-Playground (<https://edaplayground.com/>, Figura 22), que permite escribir o cargar los archivos del diseño en VHDL, y compilarlos y ejecutarlos en los servidores del sitio, obteniendo resultados incluso a nivel gráfico de las entradas y salidas declaradas.



Figura 22. Interfaz de aplicación web EDA-Playground.



Video: para recordar o profundizar el uso de VHDL, utilizando EDA-Playground, se puede ver este curso en vídeo <https://www.youtube.com/watch?v=IAiO-owoKY&list=PLju3wRXj0XQNofVFS5q0O6fRuIZruHXOx>.



Herramientas & Software de la asignatura

- **Arduino IDE**, entorno de programación para sistemas embebidos. Disponible para descargar en <https://www.arduino.cc/en/software>.
- **Tinkercad**, entorno en línea, para simular circuitos electrónicos, o modelado en 3D. Disponible en <https://www.tinkercad.com/>.
- **Circuito.io**, entorno en línea para diseñar circuitos con componentes. Disponible en <https://www.circuito.io/>.
- **EDA Playground** (opcional), simulador en línea para compilar VHDL. Disponible en <https://edaplayground.com/>.

Bibliografía

- Alexon, J. (2007). *Serial Port Complete: Ports for Embedded Systems*. Lakeview Research LLC.
- Benchimol, D. (2011). *Microcontroladores*. Buenos Aires (AR): RedUSERS.
- Floyd, T. (2006). *Fundamentos de sistemas digitales* (Novena ed.). Pearson Educación.
- FPGA4student. (2017). *A complet 8-bit Microcontroller in VHDL*. Recuperado el 01 de 01 de 2022, de <https://www.fpga4student.com/2016/12/a-complete-8-bit-microcontroller-in-vhdl.html>
- Harold, K. (2001). *Embedded Controller Hardware Design*. LLH Technology Publishing.
- Jiménez, M., Palomera, R., & Couvertier, I. (2014). *Introduction to Embedded Systems*. Springer.
- LaMeres, B. (2017). *Introduction to Logic Circuits & Logic Design with VHDL*. Springer.
- Morris Mano, M. (2003). *Diseño Digital* (Tercera ed.). Pearson Educación.
- Navia, M., Palma, V., Moreira, J., & Velez-Valarezo, R. (2022). Plataformas de hardware para nodos de IoT en aplicaciones agrícolas : un análisis orientado al consumo energético. *Revista Ibérica de Sistemas e Tecnologias de Informação*(E47).
- Sánchez-Élez, M. (2014). *Introducción a la Programación en VHDL*. Universidad Complutense de Madrid.

Este compendio recoge textualmente documentos e información de varias fuentes debidamente citadas, así como referencias elaboradas por el autor para conectar los diferentes temas.

Se lo utiliza únicamente con fines educativos.